

CONCEPTOS DE BASES DE DATOS

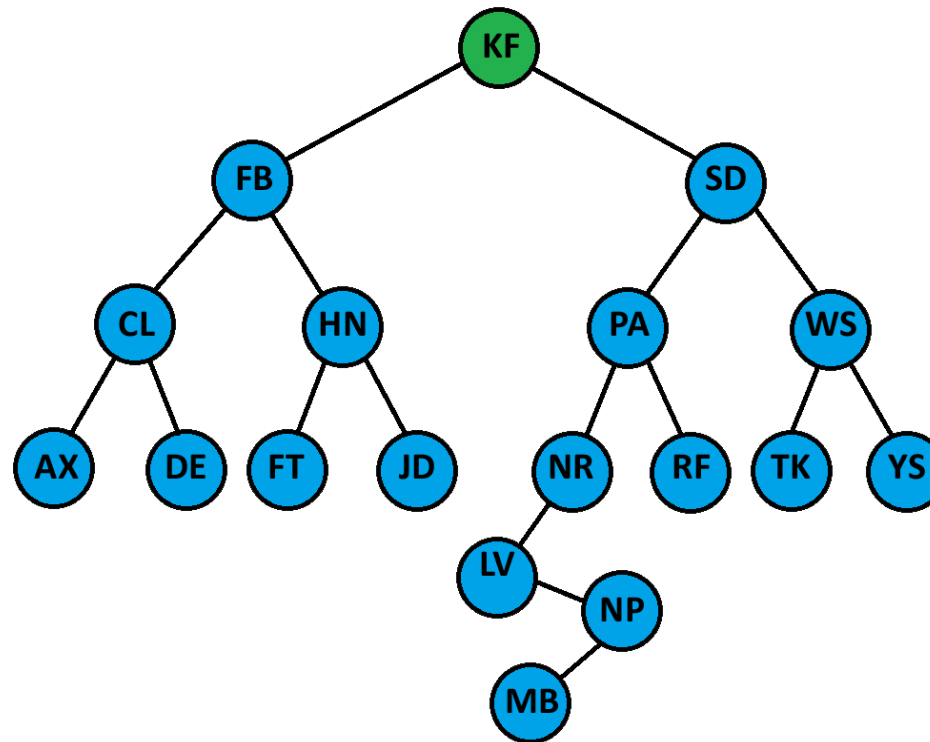
CLASE 6



Árboles

Resumen

- Un árbol **binario** se desbalancea fácilmente.



Árboles

Resumen

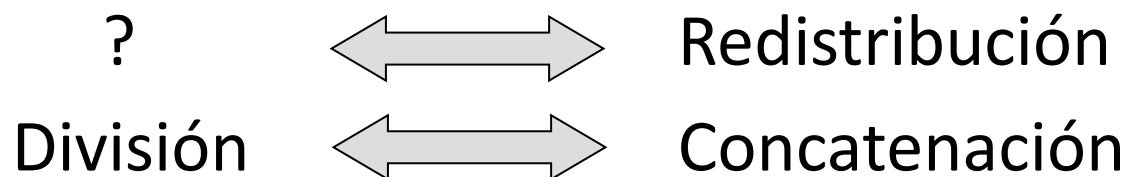
- Los árboles **balanceados en altura** proporcionan una solución aceptable al problema de **mantener un índice ordenado**.
 - Los cambios son locales (no masivos).
- Pero como son árboles binarios, son muy profundos al usarlos con muchos elementos.
 - Para 1.000.000 claves, un árbol **AVL** puede tener 28 niveles → **demasiado**.

- Los árboles **multicamino** son una buena opción que **resuelve el problema de profundidad** que tienen los árboles binarios.
- Pero para que las búsquedas realizadas tengan un costo logarítmico, se debe asegurar que el árbol esta **balanceado**.
- Para **garantizar el balanceo** de los árboles multicamino con un bajo costo computacional, su **construcción debe ser ascendente**, partiendo desde las hojas.

- Árboles B (orden M)
 - Son árboles **multicamino**.
 - Se construyen en **forma ascendente** → **balanceado a bajo costo**.
 - Cantidad de hijos por nodo:
 - Mínimo: $\lceil M/2 \rceil$ hijos (menos raíz y terminales).
 - Máximo: M hijos.
 - Nodos terminales al mismo nivel:
 - Mínimo: $\lceil M/2 \rceil - 1$ claves.
 - Máximo: $M - 1$ claves.

- Árboles B
 - **Inserción:** si el nuevo registro produce **overflow**.
 - **División del nodo.** Promoción. puede propagarse hasta **generar una nueva raíz**.
 - **Eliminación:** si el nodo queda en **underflow**.
 - **Redistribución.** Si alguno de sus **nodos adyacentes hermanos** tiene elementos suficientes, se realiza la redistribución entre dicho nodo y el que quedó en underflow.
 - **Concatenación.** Si ninguno de sus **nodos adyacentes hermanos** tiene elementos suficientes, se realiza la concatenación entre uno de ellos y el que quedó en underflow. Es posible **perder un nivel del árbol**.

- Inserción **vs** Eliminación



- La redistribución podría **posponer la creación** de nuevos nodos.
- Se pueden generar **árboles B más eficientes** en términos de utilización de espacio.

Árboles

Balanceados (B*)

- **Árbol B***: es un **árbol B especial** en donde cada nodo está lleno al menos $\frac{2}{3}$ partes (excepto la raíz).
- Propiedades de un **árbol B*** de orden **M**:
 - **Cantidad de hijos por nodo:**
 - Máximo: **M** hijos.
 - Mínimo: $\lceil \frac{2M - 1}{3} \rceil$ hijos (menos raíz y terminales).
 - Mínimo raíz: **2** hijos (o sino **ninguno**).
 - **Cantidad de claves:**
 - Máximo: **M-1** claves.
 - Mínimo: $\lceil \frac{2M - 1}{3} \rceil - 1$ claves.
 - **Nodos no terminales**: **K** hijos $\rightarrow$ **K-1** claves.
 - **Nodos terminales (hoja)**: todos están al mismo nivel.

Árboles

Operaciones (B*)

- **Búsqueda:** similar a árbol B.
- **Inserción:** redistribución / división. Tres políticas:
 - **Derecha** (de un lado):
 - Redistribuir con nodo adyacente hermano derecho (o izq. si es el último). Si esta lleno se realiza la división **usando ambos nodos**.
 - **Derecha o Izquierda** (de un lado u otro):
 - Si el nodo derecho está lleno, se intenta hacerlo con el izquierdo. Si ambos están llenos **se usa el nodo en overflow y su hermano derecho** para realizar la división.
 - **Derecha e Izquierda** (de un lado y otro):
 - Similar anterior, pero al realizar la división **se usan los tres nodos**, generando cuatro nodos que tendrán 3/4 parte llena.
- **Eliminación:** similar a árbol B (redist./concat.).

Árboles

Balanceados

- Árboles balanceados **B** y **B***
 - Los **árboles balanceados B y B*** permiten el **acceso aleatorio por clave** de forma eficiente.
 - Además, los árboles **B*** exigen que sus nodos se mantengan más llenos que los árboles **B** → **menor altura**.
 - Pero si se debe realizar un **procesamiento secuencial ordenado por clave** utilizando este tipo de árboles, ¿se puede realizar de forma eficiente?

- Archivos **secuenciales indizados**
 - Permiten dos formas de acceso:
 - **Indizado**: acceso aleatorio por clave.
 - **Secuencial**: acceso secuencial ordenado por clave.
 - Alternativas de implementación con los métodos vistos hasta ahora:
 - **Orden físico**: **ineficiente** al sufrir modificaciones.
 - **Árbol B**: rápida recuperación para acceso aleatorio pero **ineficiente** para acceso secuencial ordenado.
 - Es necesaria una solución que permita **ambos accesos de forma eficiente** → **Árboles B+**.

Árboles

Balanceados (B+)

- Los **árboles B+** constituyen una mejora sobre los árboles B:
 - Conservan la propiedad de **acceso aleatorio rápido** de los árboles B (acceso indizado).
 - Permiten además un **recorrido secuencial ordenado rápido**.

Árboles

Balanceados (B+)

- Un árbol B+ consta de dos partes:
 - **Índice:** nodos interiores.
 - En la raíz y nodos interiores se duplican las claves necesarias para definir los caminos de búsqueda.
 - **Secuencia:** nodos hojas enlazados.
 - Todas las claves se encuentran en los nodos hoja.
 - Los nodos hoja están vinculados: se obtiene una trayectoria secuencial para recorrer las claves del árbol.

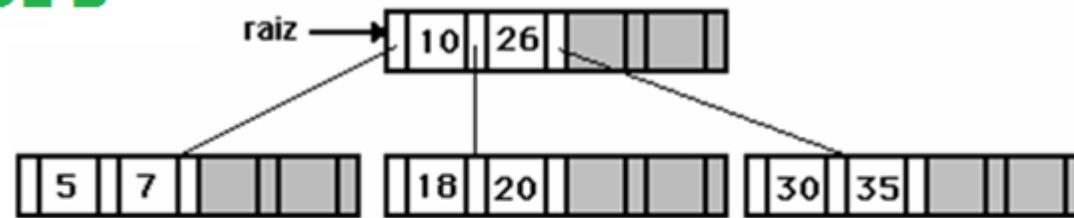
- Propiedades de un **árbol B+** de orden **M**
 - **Cantidad de hijos por nodo:**
 - **Máximo:** **M** hijos.
 - **Mínimo:** **$\lceil M/2 \rceil$** hijos (menos raíz y terminales).
 - **Mínimo raíz:** **2** hijos (o sino **ninguno**).
 - **Nodos no terminales (interiores):**
 - Son punteros a los datos: **no contienen datos**.
 - **Nodos terminales (hoja):** todos están al mismo nivel.
 - Representan un conjunto de datos y **son linkados juntos**.

Árboles

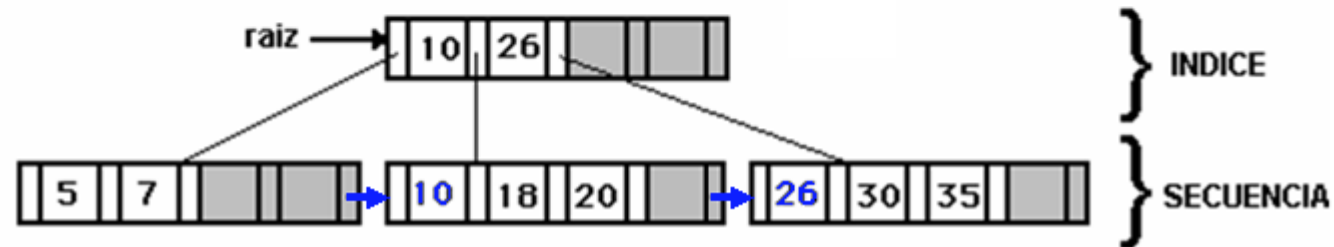
Balanceados (B+)

- Comparación

ÁRBOL B



ÁRBOL B+



Árboles

Balanceados (B+)

- Un árbol B+ **ocupa más espacio** que un árbol B equivalente, ya que además de las claves contiene las **claves-separadores**.
- Las claves del nodo raíz y nodos interiores se utilizan **únicamente como índice** para las búsquedas.

- **Búsqueda**

- **No debe detenerse** cuando se encuentre la clave en la **raíz** o en un **nodo interior**.
- La búsqueda continúa por el nodo apuntado por la **rama derecha** de dicha clave **hasta llegar a un nodo hoja**.
- Si se llega a un nodo hoja sin éxito → la clave **no se encuentra** en el archivo.

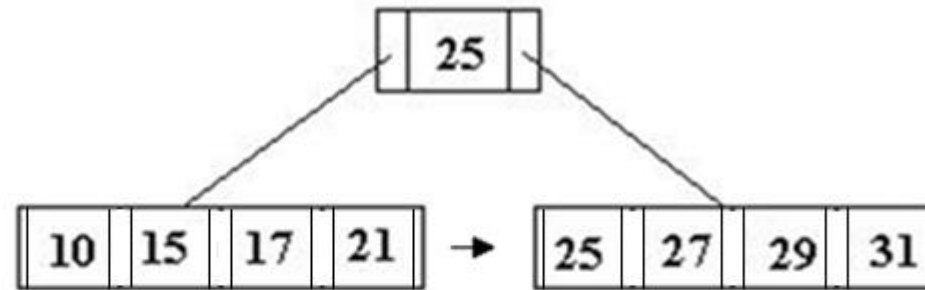
- **Insertión**

- Es **similar a la de los árboles B**, excepto cuando se desea insertar una clave en donde el nodo se encuentra lleno.
- En ese caso, el **nodo lleno se divide** en otros dos, pero ahora:
 - El primero contendrá $M/2$ claves.
 - El segundo $1+M/2$ claves, ya que lo que subirá al padre será una **copia de la clave central**.

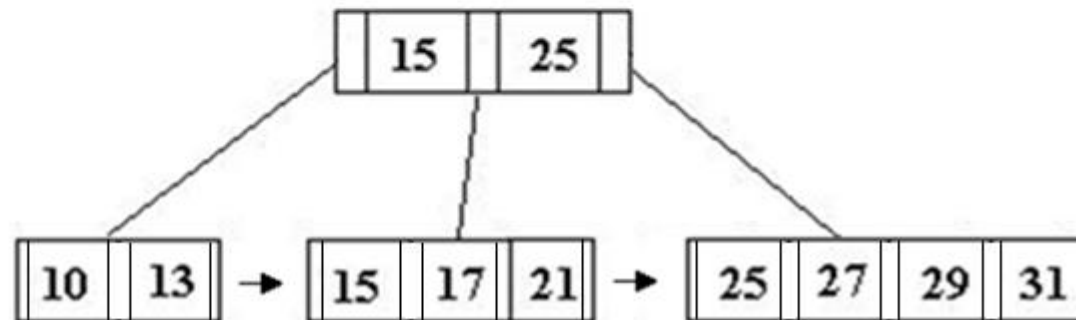
Árboles

Operaciones (B+)

- Inserción (orden $M = 5$)



AL INSERTAR LA CLAVE 13:



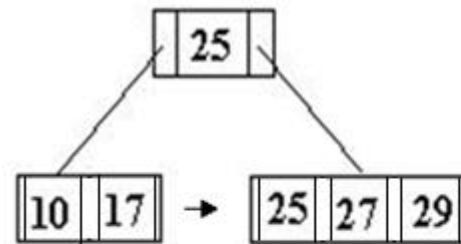
- **Eliminación**

- Todas las claves se encuentran en los nodos hoja → **más simple que en los árboles B.**
- Si al eliminar **no se produce underflow**
 - Las claves de la raíz o nodos internos **no se modifican** ya que siguen siendo un **separador válido** entre las claves de los nodos descendientes.
- Si al eliminar **se produce underflow**
 - Es necesario una **redistribución** o **fusión** de las claves, que involucra a las hojas y el índice.

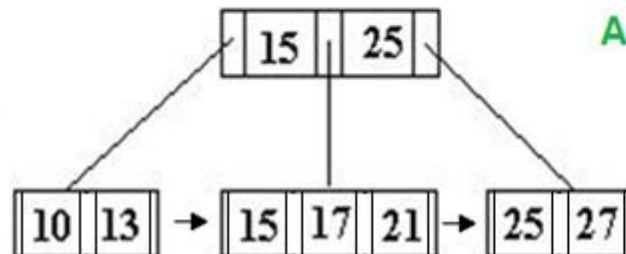
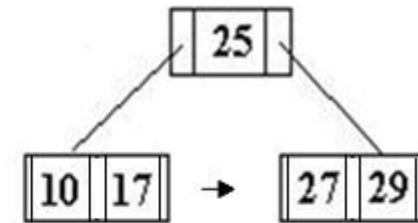
Árboles

Operaciones (B+)

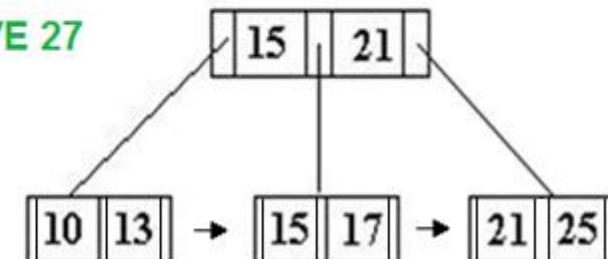
- **Eliminación** (orden $M = 6$)



AL ELIMINAR LA CLAVE 25



AL ELIMINAR LA CLAVE 27

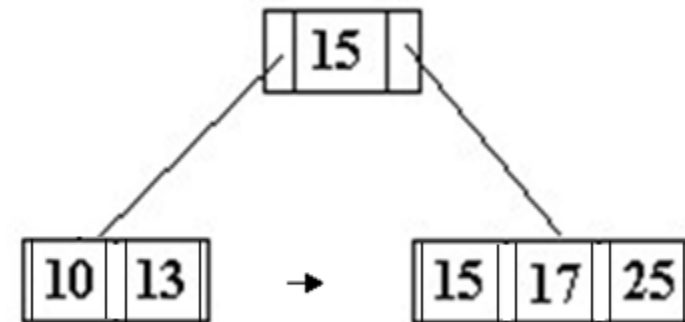
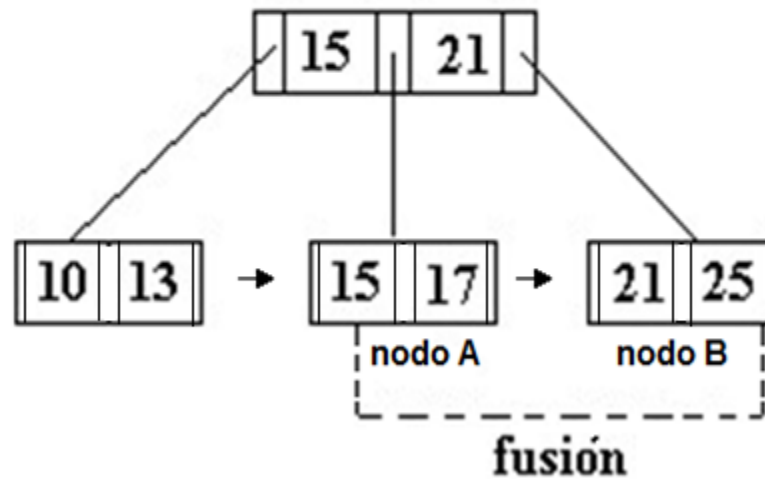


Árboles

Operaciones (B+)

- Eliminación (orden $M = 6$)

AL ELIMINAR LA CLAVE 21

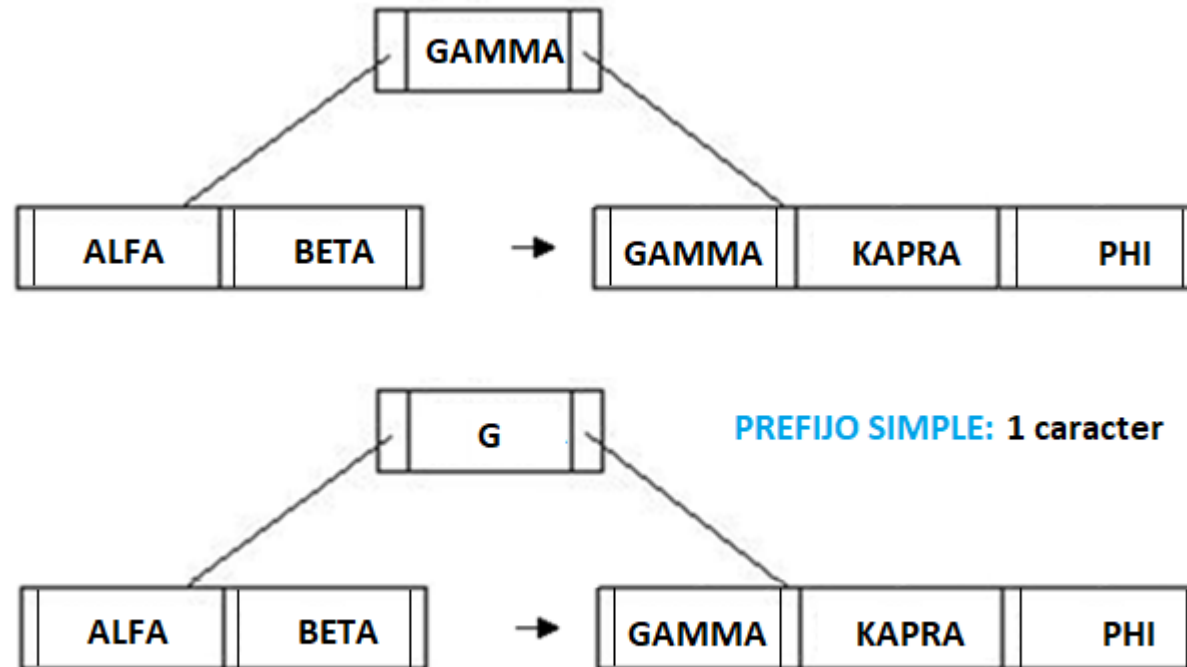


- **Árbol B+ de prefijos simples**
 - Los separadores internos son **derivados de las claves** de los registros que limitan un bloque en el conjunto de secuencia.
 - Si las claves de un índice son muy largas, es posible utilizar **separadores más cortos para ocupar menos espacio**.
 - El **árbol B+ de prefijos simples** es un árbol B+ en el cual el conjunto índice está constituido por **separadores más cortos**.

Árboles

Balanceados (B+)

- **Árbol B+ de prefijos simples**



Árboles

Balanceados

- Manejo de nodos en buffers
 - Hasta ahora se analizaron los árboles balanceados suponiendo que en todos los casos es necesario leerlo desde **memoria secundaria**.
 - Sin embargo, en la práctica los sistemas operativos y los gestores de bases de datos **utilizan buffers en memoria RAM** para conservar temporalmente los bloques más utilizados.
 - Esto permite que muchos accesos al árbol no requieran ir nuevamente a disco, **reduciendo significativamente los tiempos de respuesta**.

- Manejo de nodos en buffers
 - **Objetivo:** minimizar la cantidad de accesos a disco.
 - El SO administra una gran cantidad de buffers:
 - Estrategias de reemplazo: **LRU** (Least Recently Used).
 - El **tamaño del buffer** suele coincidir con el **tamaño del nodo**. lo que permite leer o escribir nodos completos en cada operación.
 - Los nodos más usados tienden a permanecer en memoria, especialmente:
 - Nodo raíz.
 - Nodos cercanos a la raíz.
 - Nodos consultados frecuentemente.

Árboles

Balanceados

- Manejo de nodos en buffers → Ej.
 - Cantidad de claves = 2400.
 - Cantidad de nodos = 140.
 - Altura del árbol = 3.

Cantidad de nodos en buffers en RAM	1	5	10	20
Cantidad de accesos promedio por búsqueda	3	1.71	1.42	0.97

- **Conclusiones**

- A lo largo de la evolución de las estructuras de acceso, los **árboles B** surgieron como una solución especialmente diseñada para trabajar sobre **almacenamiento secundario**.
- Su **principal ventaja** es mantener el archivo **ordenado, balanceado y con baja altura**, permitiendo búsquedas, inserciones y eliminaciones **eficientes aun con grandes volúmenes de información**.
- Cada tipo de árbol B fue pensado para **optimizar ciertos aspectos particulares**. Por eso, la elección depende del problema a resolver, del tipo de consultas que se realizarán y de las características del sistema.

- **Conclusiones**

- **Árbol B:** se utiliza cuando se busca una **estructura equilibrada y generalista**, capaz de manejar eficientemente: búsquedas puntuales, inserciones frecuentes, mantenimiento dinámico del archivo.
- **Árbol B*:** se utiliza cuando se busca **aprovechar mejor el espacio en disco**, reduciendo nodos parcialmente vacíos. Es útil cuando el almacenamiento es costoso, el volumen de datos es muy grande y se desea máxima eficiencia espacial.
- **Árbol B+:** es especialmente conveniente cuando **predominan recorridos ordenados**, búsquedas por rango, listados secuenciales, acceso intensivo a grandes volúmenes de datos.

- **Conclusiones**

- Elegir correctamente la organización del archivo puede impactar directamente en la **velocidad de respuesta**, el **uso de almacenamiento**, la **escalabilidad** del sistema y el costo de **mantenimiento**. Los árboles B ofrecen **muy buen rendimiento general**.
- Sin embargo, cuando el objetivo principal es obtener tiempos de acceso aún menores, existen otras técnicas diseñadas para **reducir aún más la cantidad de accesos necesarios**.
- Entre ellas se destacan los métodos de **dispersión** (hashing), donde la ubicación del registro puede determinarse **directamente a partir de su clave**, evitando recorridos jerárquicos.